

以下分析来自于 Huggingface Transformers

pull request add qwen2.5vl [#35569](#)

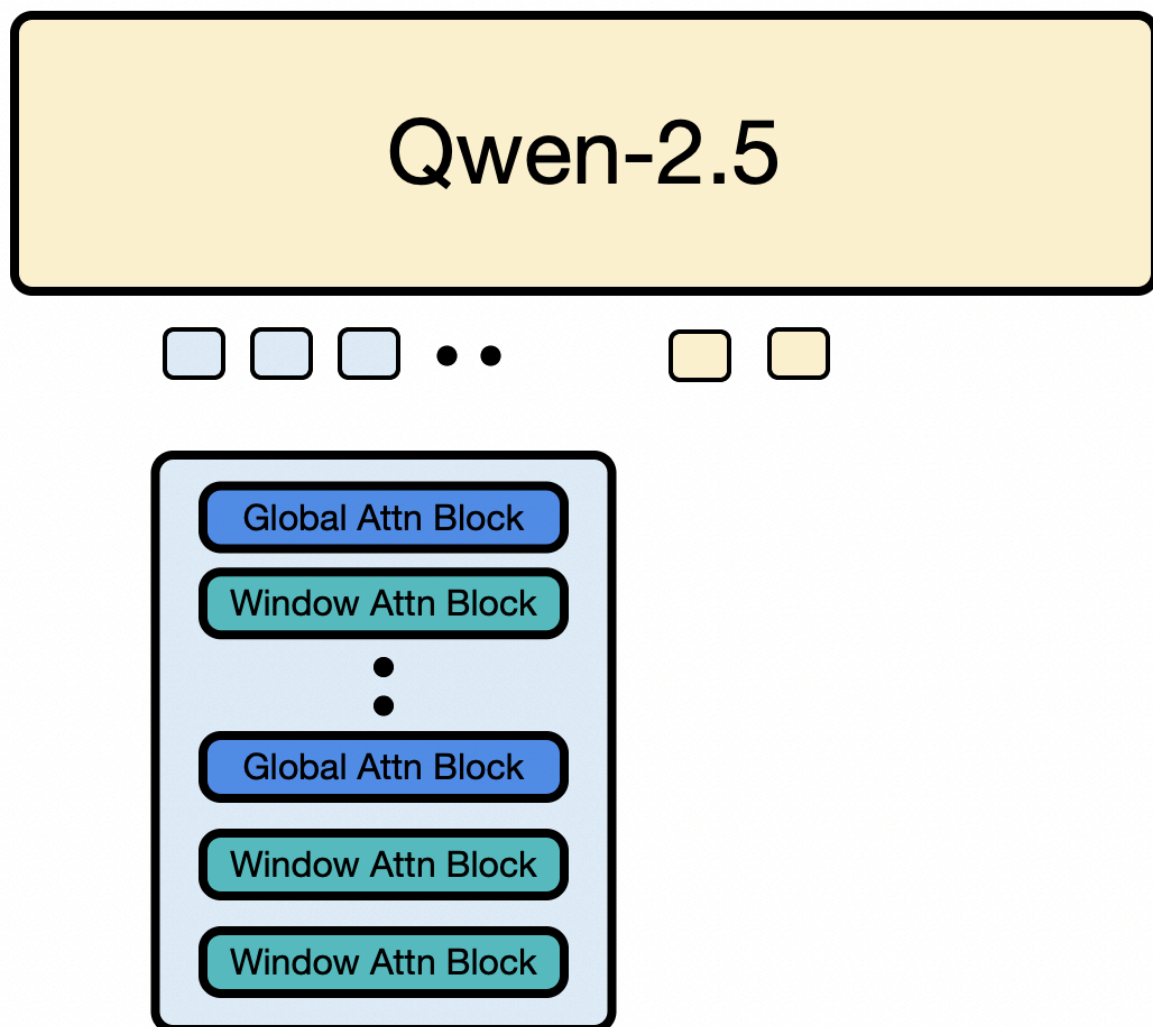
1. 模型架构

1.1 ViT 改进（重点）

1.1.1 Window Attention

即attention的计算只在window内进行，在window之间是彼此独立的。

实际操作中将Global Attention和Window Attention交错配置：



```
for layer_num, blk in enumerate(self.blocks):
    if layer_num in self.fullatt_block_indexes:
        cu_seqlens_now = cu_seqlens
```

```

        cu_seqlens_now = cu_window_seqlens if self.gradient_checkpointing and self.training:
            hidden_states = self.gradient_checkpointing_func(
                blk.__call__, hidden_states, cu_seqlens_now, rotary_pos_emb)
        else:
            hidden_states = blk(
                hidden_states,
                cu_seqlens=cu_seqlens_now,
                rotary_pos_emb=rotary_pos_emb,)

```

其中，设置7，15，23，31层为全局attention，其余为window attention：

```
fullatt_block_indexes=list(range(7, 32, 8)),
```

其中获取cu_seqlens_now（window attention分割列表）的方法参考 `get_window_index(self, grid_thw)`

1.1.2 MLP

ViT的MLP做出了更像语言模型架构的策略：

- 使用SwiGLU的FFN结构， `hidden_act="silu"`（参考[1.1.5 前馈网络 FFN 和 激活函数 Activation](#)）
- 其中的线性层增加bias，注意这里在llama这样的语言模型中是默认无偏置。

```

class Qwen2_5_VLMLP(nn.Module):
    def __init__(self, config, bias: bool = False):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.intermediate_size = config.intermediate_size
        self.gate_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=bias)
        self.up_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=bias)
        self.down_proj = nn.Linear(self.intermediate_size, self.hidden_size, bias=bias)
        self.act_fn = ACT2FN[config.hidden_act]

    def forward(self, hidden_state):
        return self.down_proj(
            self.act_fn(self.gate_proj(hidden_state)) * self.up_proj

```

```
(hidden_state)
)
```

这里也给出 [Qwen2-VL](#) 的 VisionMLP :

```
class VisionMlp(nn.Module):
    def __init__(self, dim: int, hidden_dim: int, hidden_act: str) ->
None:
        super().__init__()
        self.fc1 = nn.Linear(dim, hidden_dim)
        self.act = ACT2FN[hidden_act]
        self.fc2 = nn.Linear(hidden_dim, dim)

    def forward(self, x) -> torch.Tensor:
        return self.fc2(self.act(self.fc1(x)))
```

这里的激活函数用的是gelu , 即 `hidden_act="quick_gelu"`

1.1.3 2D-RoPE

注意这里使用的仍然是和 [Qwen2-VL](#) 一模一样的2D-RoPE :

```
head_dim = config.embed_dim // config.num_heads
self.rotary_pos_emb = VisionRotaryEmbedding(head_dim // 2)
```

即将hidden_dim均分为x轴的RoPE和y轴的RoPE , 详细解析参考 [Qwen2-VL](#)

1.2 PatchMerger

qwen2.5-VL的PatchMerger模块仅仅只是修改了LayerNorm为RMSNorm :

```
class Qwen2_5_VLPatchMerger(PatchMerger):
    def __init__(self, dim: int, context_dim: int, spatial_merge_size: int = 2) -> None:
        super().__init__(dim, context_dim, spatial_merge_size)
        self.ln_q = Qwen2RMSNorm(context_dim, eps=1e-6)
```

回顾一下qwen2-VL的PatchMerger模块 , 这里参考 [Qwen2-VL](#) , 为了控制seq_len , 在MLP之前把相邻的 2*2 个 vision token 压缩成一个(其实就是在hidden_dim上拼接) , 之后再过双层MLP (PatchMerger) 。 :

```

class PatchMerger(nn.Module):
    def __init__(self, dim: int, context_dim: int, spatial_merge_size: int = 2) -> None:
        super().__init__()
        self.hidden_size = context_dim * (spatial_merge_size**2)
        self.ln_q = LayerNorm(context_dim, eps=1e-6)
        self.mlp = nn.Sequential(
            nn.Linear(self.hidden_size, self.hidden_size),
            nn.GELU(),
            nn.Linear(self.hidden_size, dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.mlp(self.ln_q(x).view(-1, self.hidden_size))
        return x

```

1.3 LLM

语言模型基座用的是 Qwen2.5 LLM，用的是哪一版需要等report发出后解析。

1.3.1 MRoPE

MRoPE参考 [Qwen2-VL](#)，Qwen2.5-VL支持带帧率的MRoPE，参考例子中vision temporal position_ids：

Temporal (Time): 3 patches, representing different segments of the video in time.

Height: 2 patches, dividing each frame vertically.

Width: 2 patches, dividing each frame horizontally.

We also have some important parameters:

fps (Frames Per Second): The video's frame rate, set to 1. This means one frame is processed each second.

tokens per second: This is a crucial parameter. It dictates how many "time-steps" or "temporal tokens" are conceptually packed into a one-second interval of the video. In this case, we have 25 tokens per second. So each second of the video will be represented with 25 separate time points. It essentially defines the temporal granularity.

temporal_patch_size: The number of frames that compose one temporal patch. Here, it's 2 frames.

interval: The step size for the temporal position IDs, calculated as $\text{tokens_per_second} * \text{temporal_patch_size} / \text{fps}$. In this case, $25 * 2 /$

```
l = 50. This means that each temporal patch will be have a difference
of 50 in the temporal position IDs.
input ids: [V V V V V V V V V V V V T T T T T], here V is for vision.
vision temporal position_ids: [0, 0, 0, 0, 50, 50, 50, 50, 100, 100,
100, 100]
vision height position_ids: [0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
vision width position_ids: [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
text temporal position ids: [101, 102, 103, 104, 105]
text height position_ids: [101, 102, 103, 104, 105]
text width position_ids: [101, 102, 103, 104, 105]
Here we calculate the text start position_ids as the max vision posit
ion_ids plus 1.
```

具体方法的实现参考：

```
def get_rope_index(
    self,
    input_ids: torch.LongTensor,
    image_grid_thw: Optional[torch.LongTensor] = None,
    video_grid_thw: Optional[torch.LongTensor] = None,
    second_per_grid_ts: Optional[torch.Tensor] = None,
    attention_mask: Optional[torch.Tensor] = None,
)
```

2. 预训练

report 发布后解析，可以先参考[Qwen2-VL](#)的三阶段训练。